Teams & Enterprise

# Model and Integration Management

Your team can access multiple AI models and integrate Cursor with various services. This documentation covers how to control which models are available, manage MCP server trust, and set up integrations with tools like Slack, GitHub, and Linear.

## Model access control

Enterprise teams can control which AI models team members can use, contact sales to get access. This helps manage costs, ensure appropriate usage, and comply with organizational policies.

Model access controls are configured through the team dashboard. Navigate to Settings and look for "Model Access Control" (Enterprise only).

### How enterprise model rollout works

When new models become available, Cursor doesn't immediately enable them for all enterprise teams.

Instead, Enterprise teams can opt in to new models for their organization.

See Models for the current list of available models.

## Restrict personal API keys (BYOK controls)

Enterprise teams can prevent team members from using their own API keys with third-party providers (OpenAI, Anthropic, Azure, AWS Bedrock) in Cursor. All usage goes through Cursor's included models and usage pool.

Configure this in the team dashboard under Settings (Enterprise only).

## MCP server trust management

The Model Context Protocol (MCP) lets you connect external tools and data sources to Cursor. MCP servers can:

- Read files from external systems
- Execute operations on your behalf
- Access databases and APIs
- Integrate with third-party services

MCP servers are designed and implemented by external vendors, not Cursor. We work with partners to provide a vetted marketplace of trusted servers, but you should review each server's capabilities and permissions before enabling it for your team.

Because MCP servers have significant capabilities, you need to manage which servers your team can use.

### MCP Allowlist

Enterprise teams can control which MCP servers team members are allowed to use. Configure this in the team dashboard under "MCP Configuration" (Enterprise only).

You can also distribute `~/.cursor/permissions.json` through MDM to set the per-user MCP auto-run allowlist from a managed file.

In that file, `mcpAllowlist` must be a JSON array of strings using `server:tool` syntax:

| Entry | Meaning |
| --- | --- |
| `server:tool` | One specific tool on one specific MCP server |
| `server:*` | All tools from one MCP server |

| | |
|---|---|
| `*:tool` | One tool name from any MCP server |
| `*:*` | All MCP tools |

Cursor resolves the effective MCP allowlist in this order:

1. Team dashboard or other admin-controlled settings
2. `~/.cursor/permissions.json`
3. The MCP allowlist in editor settings and inline **Add to allowlist**

Higher-priority sources replace lower-priority ones. They do not merge.

When an allowlist is active, only servers matching an allowlist entry can run. Servers that don't match are blocked.

Adding a server to the allowlist does not push it to users' machines. Team members still need to configure the server in their own Cursor settings.

All allowlist entries support wildcards using `*` to match any sequence of characters.

### Command-based servers (stdio)

For local MCP servers configured with `command` and `args`, the allowlist matches against the **full command string**: the `command` value and all `args` values joined with spaces.

Given this `mcp.json` config:

```
{
  "mcpServers": {
    "my-tool": {
      "command": "npx",
      "args": ["-y", "@acme/mcp-tool@latest"]
    }
  }
}
```

The full command string is `npx -y @acme/mcp-tool@latest`. On most systems, the shell resolves `npx` to a full path like `/usr/local/bin/npx` or `/opt/homebrew/bin/npx`, so the actual string becomes `/usr/local/bin/npx -y @acme/mcp-tool@latest`.

Use a leading `*` wildcard to match regardless of the install path:

| Allowlist entry | Matches |
|---|---|
| `*npx -y @acme/mcp-tool@latest` | `npx` at any path, with these exact arguments |
| `/usr/local/bin/npx -y @acme/mcp-tool@latest` | Only this exact path |
| `*npx -y @acme/*` | Any `@acme`-scoped MCP package |
| `*python */scripts/mcp-server.py*` | A Python server at any matching path, with any trailing arguments |

### URL-based servers (HTTP/SSE)

For remote MCP servers configured with `url`, the allowlist matches against the URL.

Given this `mcp.json` config:

```
{
  "mcpServers": {
    "acme-tools": {
      "url": "https://mcp.acme.com/sse"
    }
  }
```

```
}
```

The allowlist entry matches against the full URL `https://mcp.acme.com/sse`:

| Allowlist entry | Matches |
| --- | --- |
| `https://mcp.acme.com/sse` | This exact URL |
| `https://*.acme.com/*` | Any subdomain and path under `acme.com` |
| `https://mcp.acme.com/*` | Any path on this host |

## Git repository blocklist

You can prevent Cursor from accessing specific repositories.

Add repository URLs or patterns in the team dashboard under "Repository Blocklist" (Enterprise only). Cursor will refuse to index or work with blocked repositories.

## Integration: Slack

The Slack integration enables Cloud Agents to run directly from Slack. Team members can mention `@cursor` with a prompt and get automated code changes delivered as pull requests.

Cursor requires permissions to read messages, post responses, and access channel metadata. See the Slack integration documentation for the full list.

See Slack integration for detailed setup and usage instructions.

## Integration: GitHub, GHES, and GitLab

Connect Cursor to your version control system to work with Cloud Agents.

Cursor requires read access to repositories and write access to create PRs. You control which repositories the Cursor app can access.

See GitHub integration for setup.

## Integration: Linear

Connect Linear to start Cloud Agents from issues.

Cursor requires read access to issues and write access to update issue status.

See Linear integration for details.

Model controls are available on the Enterprise plan
Contact our team to learn about model restrictions and MCP management.
Contact Sales